

Le modèle de Bell et La Padula

Le modèle de Bell et LaPadula, développé par David Elliott Bell et Len LaPadula pour le département de la défense américaine, formalise la sécurité multi-niveaux et met l'accent sur la confidentialité des données dans divers systèmes informatiques tels que les systèmes d'exploitation et les systèmes de gestion de bases de données.

À la différence du modèle proposé par Lampson, le modèle de Bell et LaPadula se concentre sur les flux d'information plutôt que sur l'accès direct aux ressources. Il est basé sur les concepts de sujets (utilisateurs ou processus), d'objets (ressources ou données) et de modes d'accès, mais il exprime les objectifs de sécurité en termes de circulation et de contrôle des informations plutôt qu'en termes d'accès.

Dans ce modèle, les droits d'accès dépendent des classifications attribuées aux objets et des habilitations attribuées aux sujets. Les classifications représentent les niveaux de sensibilité des informations et les habilitations représentent les niveaux de confiance ou d'autorisation des sujets à quelles ressources.

Le modèle de Bell et La Padula (2)

Les règles obligatoires imposées par le modèle de Bell et LaPadula visent à prévenir la circulation des informations d'un niveau de sécurité élevé vers un niveau de sécurité plus bas. Ces règles assurent que les informations classifiées à un niveau de sécurité élevé ne peuvent être transférées vers des sujets ou des objets classifiés à un niveau de sécurité inférieur, préservant ainsi la confidentialité des données.

De plus, le modèle introduit la notion de niveaux de sécurité formant un ensemble partiellement ordonné ($L; \leq$), où les niveaux de sécurité sont organisés hiérarchiquement pour déterminer les autorisations d'accès en fonction de la sensibilité des informations. Cela permet de définir clairement les restrictions et les autorisations d'accès pour chaque niveau de sécurité, contribuant ainsi à renforcer la sécurité et la confidentialité des systèmes informatiques.

Dans le modèle de Bell et LaPadula, la sécurité est abordée sous l'angle de la circulation et du contrôle des informations plutôt que de l'accès direct aux ressources. Cela signifie que le modèle se concentre sur la manière dont les informations se déplacent à travers le système et sur la manière dont elles sont contrôlées pour garantir la confidentialité et l'intégrité des données, plutôt que sur qui peut accéder.

Le modèle de Bell et La Padula (3)

Dans le modèle de contrôle d'accès de Bell-LaPadula, chaque sujet et chaque objet du système d'information se voient attribuer un niveau de sécurité. Ce niveau de sécurité est crucial pour déterminer les autorisations d'accès des sujets aux objets. Voici quelques points clés pour comprendre ce modèle :

1. Niveau d'habilitation et niveau de classification: Chaque sujet (utilisateur ou processus) se voit attribuer un niveau d'habilitation, tandis que chaque objet (donnée ou ressource) se voit attribuer un niveau de classification. Ces niveaux sont utilisés pour évaluer les autorisations d'accès.

2. Fonctions d'habilitation et de classification: Les fonctions f_s et f_o assignent respectivement les niveaux d'habilitation et de classification aux sujets et aux objets. Ces fonctions déterminent les niveaux de sécurité associés à chaque sujet et à chaque objet dans le système.

La fonction d'habilitation est notée $f_s: S \rightarrow L$ et celle de classification $f_o: O \rightarrow L$.

Le modèle de Bell et La Padula (4)

3. **Propriété de tranquillité forte:** Une caractéristique importante du modèle est la propriété de tranquillité forte, qui stipule que les niveaux de sécurité attribués aux sujets et aux objets ne peuvent pas être modifiés une fois qu'ils ont été établis. Cela garantit la stabilité et la prévisibilité du système de contrôle d'accès.

4. **Propriété de sécurité simple :** La propriété de sécurité simple, également connue sous le nom de principe "no read up", est une règle fondamentale du modèle Bell-LaPadula. Elle stipule qu'un sujet ne peut accéder à un objet que si le niveau d'habilitation du sujet est supérieur ou égal au niveau de classification de l'objet. En d'autres termes, un sujet ne peut pas lire des informations classifiées à un niveau de sécurité plus élevé que le sien. Cette règle vise à prévenir la divulgation non autorisée d'informations sensibles et à garantir la confidentialité des données.

Formellement : $\forall s \in S; \forall o \in O; (s; o; \text{read}) \in M \longrightarrow f_O(o) \leq f_S(s)$

Top secret

O

S

Secret

O

S

Confidentiel

S

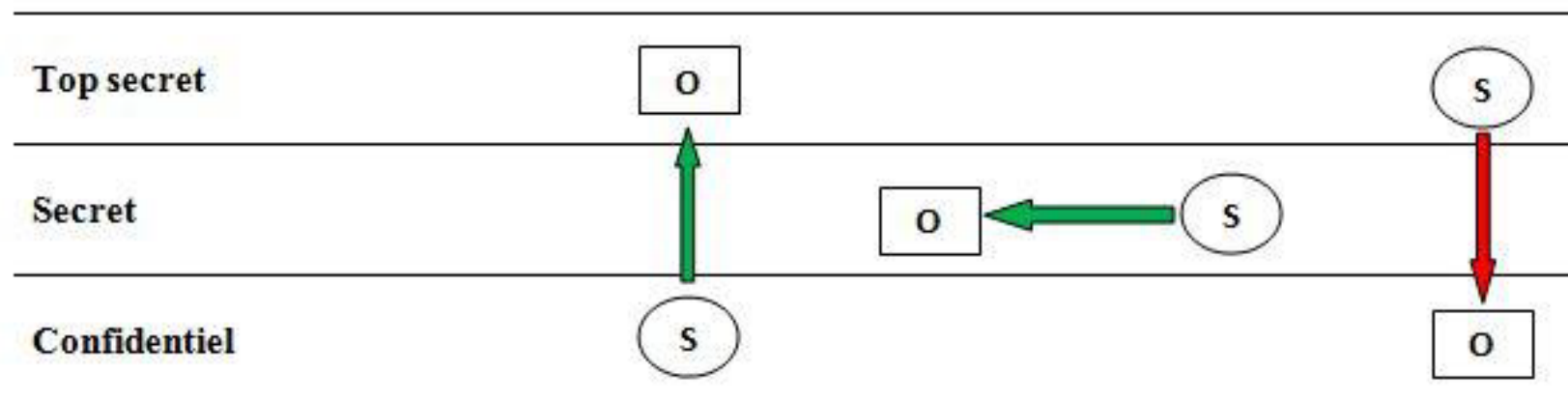
O



Le modèle de Bell et La Padula (5)

5. **La propriété étoile:** La propriété étoile du modèle de sécurité Bell-LaPadula établit une règle fondamentale dans la gestion de la confidentialité des informations : un sujet ne peut écrire dans un objet que si son niveau d'accréditation est inférieur ou égal au niveau de classification de cet objet. Cette règle est résumée par l'expression "ne pas écrire en bas". En d'autres termes, elle garantit que les utilisateurs ne peuvent pas accéder à des informations plus sensibles que leur niveau d'autorisation ne le permet, évitant ainsi les fuites de données et préservant l'intégrité du système.

Formellement : $\forall s \in S; \forall o \in O; (s; o; \text{write}) \in M \Rightarrow f_S(s) \leq f_O(o)$.



Variantes du modèle de Bell et La Padula

Dans le domaine de la sécurité informatique, des variantes du modèle Bell-LaPadula ont été introduites pour offrir plus de souplesse et d'expressivité. Ces variantes traduisent les notions d'observation et de modification de l'information non seulement par les opérations élémentaires de lecture et d'écriture, mais également par d'autres actions telles que l'exécution et l'ajout.

L'action "**Exécuter**" permet un accès à l'information sans la modifier ni l'observer, par exemple dans le cas où un programme est simplement exécuté sans lire ni écrire de données.

L'action "**Lire**" permet d'observer l'information sans la modifier, ce qui peut être utile dans des scénarios où la simple consultation des données est nécessaire.

L'action "**Ajouter**" (ou "append") permet de modifier l'information sans l'observer, par exemple lors de l'ajout de données dans des fichiers de journalisation d'audit ou lors de l'ajout de chemins dans des fichiers de configuration lors de l'installation de logiciels.

Enfin, l'action "**Écrire**" englobe à la fois l'observation et la modification de l'information, permettant ainsi un accès complet aux données.

Ces différentes actions offrent une granularité plus fine dans la gestion des autorisations d'accès aux informations, permettant ainsi de mieux répondre aux besoins spécifiques des applications et des environnements de sécurité.

Le modèle de sécurité peut être représenté par une machine à états où chaque état est défini par un triplet (S, O, M) , où :

S : représente un ensemble de sujets, qui sont les entités (utilisateurs, processus, etc.) qui accèdent aux objets.

O : représente un ensemble d'objets, qui sont les ressources (fichiers, bases de données, etc.) auxquelles les sujets accèdent.

A : représente un ensemble d'opérations d'accès, qui inclut des actions telles que exécuter, lire, ajouter et écrire.

un ensemble de niveaux de sécurité muni d'une relation d'ordre partiel, qui définit la hiérarchie des niveaux de sécurité.

B : représente l'ensemble de tous les états possibles de la machine à états.

M : représente l'ensemble des matrices (M_{so}) où $s \in S$ et $o \in O$. Une matrice M_{so} spécifie les autorisations d'accès des sujets aux objets, indiquant quelles opérations chaque sujet est autorisé à effectuer sur chaque objet.

$F \subset L_s^* \times L_o^* \times L_o$: représente l'ensemble des niveaux de sécurité dans un état donné. f_s est le niveau de sécurité maximal qu'un sujet peut avoir, f_c est le niveau de sécurité actuel de chaque sujet et f_o est la classification de l'objet.

Les deux règles de contrôle d'accès dans le contexte de la sécurité informatique sont les suivantes :

La propriété simple : Cette règle stipule qu'un état est considéré comme sécurisé si, pour chaque sujet s qui observe un objet o , le niveau de sécurité maximal du sujet ($fs(s)$) domine le niveau de sécurité de l'objet ($fo(o)$). Formellement, un état satisfait la propriété simple si pour chaque triplet (s, o, a) dans l'ensemble des états possibles (b), on a $fo(o) \leq fs(s)$. En d'autres termes, un sujet ne peut observer un objet que si son niveau de sécurité est au moins aussi élevé que celui de l'objet.

$$\forall (s, o, a) \in b, \Rightarrow fo(o) \leq fs(s)$$

La propriété étoile : Cette règle indique qu'un état est considéré comme sécurisé si, pour chaque sujet s qui modifie un objet o , le niveau de sécurité de l'objet ($fo(o)$) domine le niveau courant de sécurité du sujet ($fc(s)$). Formellement, un état est considéré comme sécurisé si pour chaque triplet (s, o, a) dans l'ensemble des états possibles (b), on a $fc(s) \leq fo(o)$. En d'autres termes, un sujet ne peut modifier un objet que si le niveau de sécurité de l'objet est au moins aussi élevé que le niveau de sécurité actuel du sujet.

$$\forall (s, o, a) \in b, \Rightarrow fc(s) \leq fo(o)$$

En résumé, ces deux règles de contrôle d'accès garantissent que les sujets ne peuvent observer ou modifier des objets que s'ils ont un niveau de sécurité approprié, assurant ainsi l'intégrité et la confidentialité des données dans les systèmes informatiques.

Le modèle de Biba 1977

Les politiques obligatoires sont des règles de sécurité développées pour maintenir l'intégrité des données dans les systèmes informatiques. Elles appliquent à l'intégrité un modèle similaire à celui de Bell-LaPadula, qui est utilisé pour la confidentialité.

Selon ces politiques, chaque sujet se voit attribuer un niveau d'intégrité (is(s)), correspondant à la confiance accordée à ce sujet, et chaque objet possède un niveau d'intégrité (io(o)), correspondant au niveau d'intégrité du sujet qui l'a créé. Les objectifs de sécurité de cette politique visent à :

Interdire toute propagation d'information d'un objet situé à un certain niveau d'intégrité vers un objet situé à un niveau d'intégrité supérieur. Cela garantit que les données ne peuvent pas être modifiées ou altérées par des sujets de confiance inférieure, préservant ainsi leur intégrité.

Interdire à tout sujet situé à un certain niveau d'intégrité de modifier un objet possédant un niveau d'intégrité supérieur. Cela garantit que les sujets de confiance inférieure ne peuvent pas altérer les données créées par des sujets de confiance supérieure, préservant ainsi leur intégrité et évitant toute altération ou corruption non autorisée.

Le modèle de Biba 1977 (2)

Le schéma d'autorisation en matière de sécurité découle de la recherche de certaines propriétés, en tenant compte des labels d'intégrité et du regroupement des opérations en trois classes : observation, modification et invocation.

Un sujet ne peut modifier un objet que si le label d'intégrité du sujet domine le label d'intégrité de l'objet :

$$(si, oj, modifier) \Rightarrow io(oj) \leq is(si).$$

Cette règle garantit que seuls les sujets dont le niveau d'intégrité est au moins aussi élevé que celui de l'objet peuvent modifier cet objet, préservant ainsi l'intégrité des données.

Un sujet ne peut observer un objet que si le label d'intégrité de l'objet domine le label d'intégrité du sujet :

$$(si, oj, observer) \Rightarrow is(si) \leq io(oj).$$

Cette règle assure que seuls les sujets dont le niveau d'intégrité est inférieur ou égal à celui de l'objet peuvent observer cet objet, évitant ainsi toute divulgation non autorisée d'informations sensibles.

Un sujet ne peut invoquer un sujet j que si le label d'intégrité de si domine le label d'intégrité de sj :

$$(si, sj, invoquer) \Rightarrow is(sj) \leq is(si).$$

Cette règle garantit que seuls les sujets dont le niveau d'intégrité est au moins aussi élevé que celui du sujet invoqué peuvent effectuer cette action, préservant ainsi la sécurité du système en limitant les accès aux sujets de confiance suffisante.

Le modèle Clark et Wilson

Dans l'environnement commercial, la prévention contre les modifications non autorisées de l'information, notamment contre les fraudes et les erreurs, est d'une importance capitale. Même si un utilisateur est autorisé à manipuler certaines données, cela ne doit pas entraîner de perte ou de falsification des actifs, tels que les capitaux et les informations.

La politique de Clarke-Wilson repose sur deux principes bien connus :

Transactions bien formées : Ce principe stipule que les utilisateurs n'ont pas le droit de manipuler les données de manière arbitraire, mais seulement à travers des procédures de transformation spécifiques qui préservent l'intégrité des données. Les actions des utilisateurs doivent suivre des protocoles prédéfinis et autorisés pour garantir que les données restent exactes et fiables.

Séparation des pouvoirs : Aussi connue sous le nom de "separation of duty", ce mécanisme de contrôle des fraudes et des erreurs est basé sur la répartition des opérations entre plusieurs parties et l'attribution de droits différents mais complémentaires à différentes catégories de personnes. En séparant les responsabilités et les autorisations entre plusieurs individus ou entités, ce principe réduit les risques de fraudes et d'erreurs en obligeant plusieurs personnes à s'impliquer dans un processus de validation ou de transaction.

Le modèle Clark et Wilson (2)

Dans ce modèle, plusieurs concepts clés doivent être définis pour assurer la sécurité des données :

1.Les données contraintes (CDI) : Ces données représentent les éléments de données dont nous voulons préserver l'intégrité. Elles sont soumises à des contraintes et à des mécanismes de protection pour garantir qu'elles ne sont pas altérées de manière non autorisée.

2.Les données non contraintes (UDI) : Ces données représentent les éléments dont l'intégrité n'est pas garantie et qui peuvent être manipulés de manière arbitraire. Elles ne sont pas soumises aux mêmes contraintes de protection que les CDI.

3.Procédures pour la vérification de l'intégrité (IVP) : Ces procédures sont utilisées pour confirmer que tous les CDI dans les systèmes sont conformes aux spécifications d'intégrité au moment où les IVP sont exécutées. Elles permettent de s'assurer que les données contraintes restent intactes et non altérées.

4.Procédures de transformation (TP) : Correspondant au concept de transactions bien formées, ces procédures visent à changer les CDI d'un état valide à un autre état valide. Elles permettent des modifications contrôlées et autorisées des données contraintes tout en préservant leur intégrité et leur fiabilité.

Le modèle Clark et Wilson (3)

Dans le modèle de sécurité de Clarke-Wilson, les règles appliquées dans leur politique de sécurité sont définies de la manière suivante :

L'ensemble des utilisateurs, noté $U=\{u_1, u_2, \dots\}$.

L'ensemble des données contraintes, $CDI=\{CDI_1, CDI_2, \dots\}$.

L'ensemble des données non contraintes, $UDI=\{UDI_1, UDI_2, \dots\}$.

L'ensemble des opérations de vérification de l'intégrité des données, $IVP=\{IVP_1, IVP_2, \dots\}$. Ainsi, les CDI sont certifiées par des IVP.

L'ensemble des opérations de transformation des données (Transformation Procedures), noté $TP=\{TP_1, TP_2, \dots\}$. Les CDI ne peuvent être manipulées que par des TP.

Une relation R_c liant chaque opération de transformation TP_i à un sous-ensemble c de CDI ; R_c précise les données que l'opération peut manipuler.

Une relation R_u liant chaque utilisateur u et chaque opération de transformation TP_i à un sous-ensemble de CDI ; R_u précise les données contraintes qu'un utilisateur est autorisé à manipuler (par le biais de TP_i).

Le modèle Clark et Wilson (4)

Les règles obligatoires du modèle de Clarke-Wilson imposent plusieurs contraintes :

Les CDI doivent être dans un état valide, vérifié par les IVP. Cela garantit que les données contraintes sont toujours fiables et conformes aux spécifications d'intégrité.

Toutes les opérations de TP doivent être certifiées. Si une donnée contrainte est valide avant l'exécution d'une opération TP_i , elle doit rester valide après l'exécution de TP_i . De plus, les opérations de TP ne peuvent être effectuées que sur des données contraintes spécifiées par R_c .

Chaque utilisateur n'effectue des opérations de transformations sur des données contraintes que si celles-ci lui sont associées par R_u . Cela garantit que chaque utilisateur ne peut manipuler que les données auxquelles il est autorisé.

Le modèle Clark et Wilson (5)

La relation Ru doit être certifiée afin de refléter les besoins de la séparation des pouvoirs. Cela assure que les droits et les autorisations des utilisateurs sont correctement définis et respectés.

Le système doit authentifier l'identité de chaque utilisateur souhaitant exécuter une opération de TP. Cela garantit que seuls les utilisateurs autorisés peuvent effectuer des opérations sur les données.

Avant d'être exécutées, les opérations de TP doivent enregistrer leurs identifiants dans les journaux d'audit, qui sont considérés comme des données contraintes. Cela permet de garder une trace des actions effectuées sur les données et de garantir la traçabilité des opérations.

Chaque opération TP qui accepte une donnée UDI en entrée doit garantir que, quelle que soit la valeur de UDI, elle n'effectue que des transformations conduisant à une donnée contrainte valide (CDIk) ou rejette UDI si elle ne peut pas être transformée de manière valide. Cela assure que seules les données contraintes sont manipulées de manière sécurisée dans le système.

Le modèle Muraille de chine

Le modèle de Brewer et Nash, également connu sous le nom de modèle de la muraille de Chine, est un modèle de contrôle d'accès qui fournit un contrôle d'accès à l'information pouvant évoluer dynamiquement.

Ce modèle a été développé pour réduire les conflits d'intérêts dans des organisations commerciales et assurer la propriété de la confidentialité. Un conflit d'intérêts (COI) survient lorsqu'un sujet accède aux informations confidentielles de deux organisations en compétition, permettant ainsi un flux d'information d'une organisation vers une autre.

Les concepts clés de ce modèle incluent :

- **Objet** : qui est un élément appartenant à une organisation.
- **Organisation** : qui dispose de plusieurs objets utilisés dans le cadre de ses services.
- **Classe de conflits d'intérêts** : qui regroupe des organisations qui sont en concurrence les unes avec les autres.

Le modèle Muraille de chine (2)

Dans ce modèle, le système classe l'information en trois niveaux hiérarchiques :

1.Au niveau le plus bas, on trouve les données simples, telles que les fichiers, qui sont les objets du système.

2.Au niveau intermédiaire, les objets sont regroupés selon la compagnie à laquelle ils appartiennent, formant des ensembles de données (CD).

3.Au niveau le plus élevé, les compagnies sont regroupées en fonction de leur compétitivité ou de leur absence de compétitivité. Cela forme les classes de conflit d'intérêt (COD) mentionnées précédemment.

Les auteurs introduisent une matrice N qui contient l'historique des accès précédemment effectués. Il s'agit de placer les sujets en lignes et les objets en colonnes, où chaque case contient un booléen indiquant si un accès a déjà eu lieu pour le couple (sujet, objet) correspondant. Ainsi, deux règles régissent les accès entre sujets et objets :

•**Simple property** : L'accès en lecture à un objet est autorisé si :

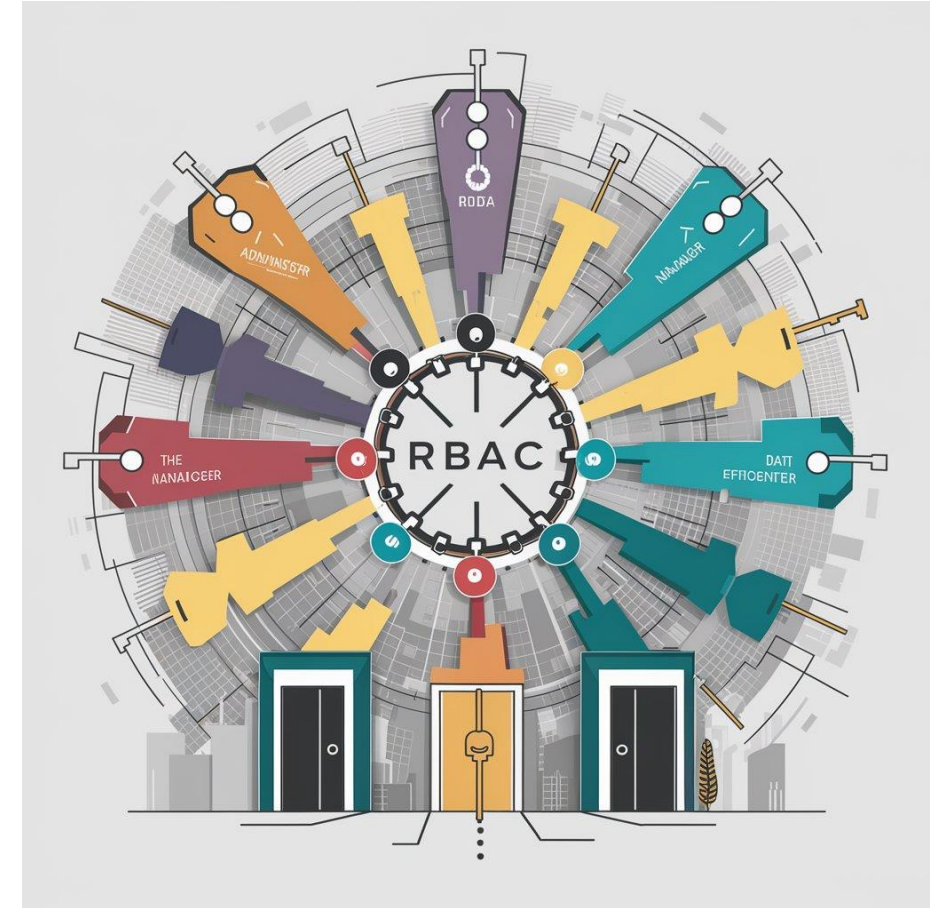
- Il appartient à la même compagnie qu'un objet déjà accédé.
- Ou s'il appartient à une autre classe de conflit d'intérêt que les objets déjà en accès.

•**Étoile (*)-property** : L'accès en écriture à un objet est autorisé si :

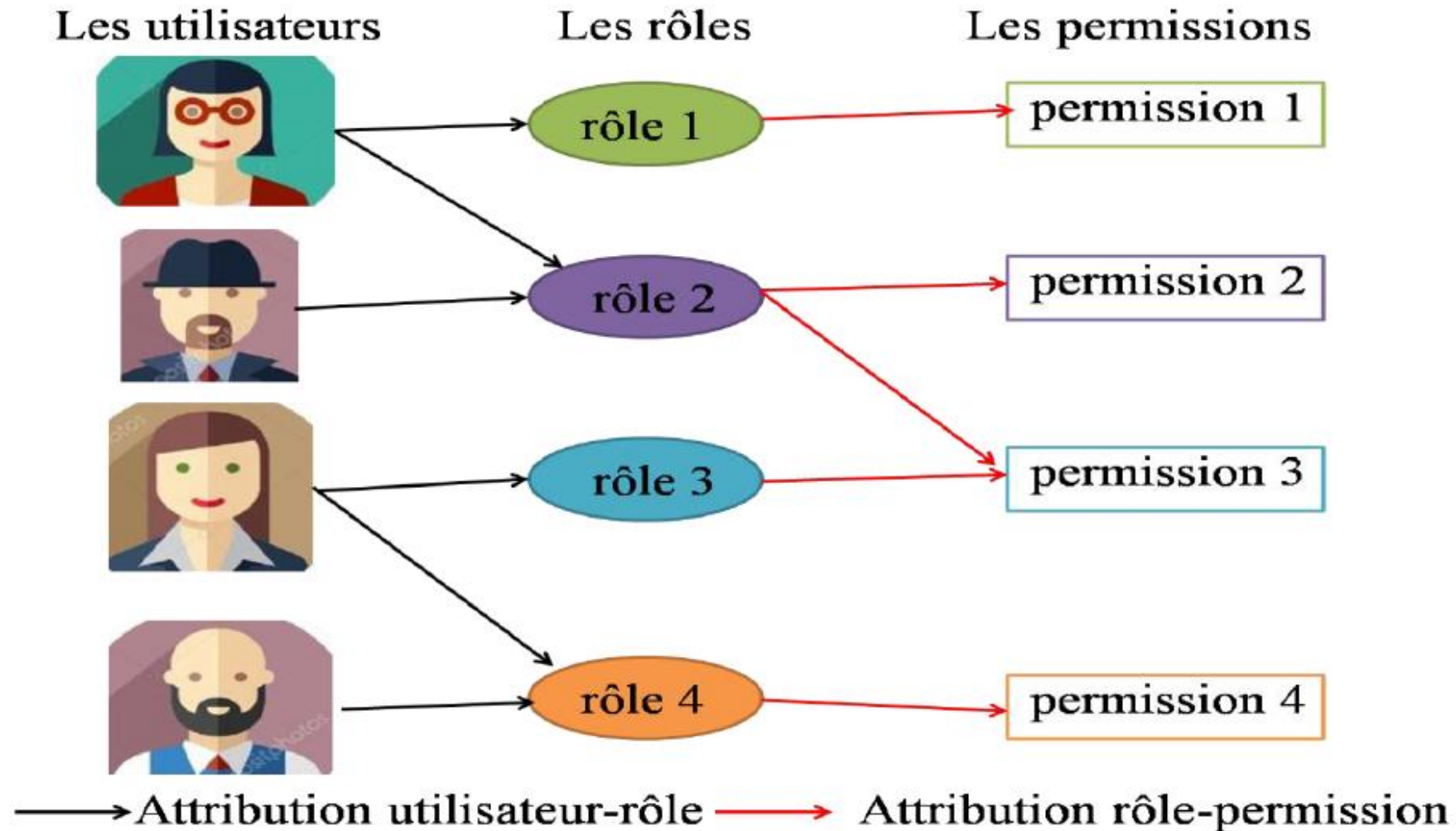
- L'accès est autorisé par la règle de sécurité simple.
- Et si le sujet ne peut pas lire aucun objet d'une autre compagnie contenant des informations non stérilisées.

Le modèle de contrôle d'accès basé sur le rôle (RBAC)

Il a été présenté comme une alternative au DAC et au MAC. Le RBAC (pour Contrôle d'Accès Basé sur les Rôles) [Ferraiolo&Kuhn,1992],[Sandhu et al.,1996]) est un modèle de contrôle d'accès qui se concentre sur les rôles. Chaque rôle représente de manière abstraite une fonction identifiée dans l'organisation, telle que chef de service, ingénieur d'étude, etc. À chaque rôle sont associées des permissions, qui sont représentées par des droits d'accès. Une permission est un ensemble de droits correspondant aux tâches pouvant être effectuées par un rôle.



Le modèle de contrôle d'accès basé sur le rôle (RBAC) (2)



Le modèle de contrôle d'accès basé sur le rôle (RBAC) (3)

Par exemple: soit l'utilisateur U1 qui possède le rôle « directeur d'hôpital », les permissions qui sont attribuées à ce rôle sont : (gérer, soigner, acheter). De plus, gérer l'accès à un système utilisant ce modèle nécessite le respect de certaines politiques de contrôles d'accès qui sont faites à base de rôle, par exemple : Pour qu'un utilisateur puisse accéder aux informations médicales d'un patient de l'hôpital, il doit avoir le rôle « médecin ». De cette manière l'utilisation du rôle offre une indépendance logique dans la spécification des autorisations d'utilisateur. En outre, cette indépendance simplifie la gestion de la politique de sécurité car lorsqu'un nouvel utilisateur rejoint

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (4)

En 1992, Feraïolo et Kuhn ont développé le premier modèle RBAC [FK92].

En 1995, ils l'ont bien raffiné en définissant tous les termes du modèle RBAC et ont validé son utilisation et sa faisabilité [FCK95].

En 1996, Sandhu et al. [SCFY96] proposent la famille la plus connue des modèles RBAC.

En 2001, NIST a proposé un modèle standard pour RBAC.

En 2004, ANSI (The American National Standard Institute) a proposé le standard ANSI RBAC₃ en se basant sur [SFK00] et [FSG+01].

Ce standard définit un modèle RBAC de référence et fournit les caractéristiques que tout système RBAC doit avoir.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (5)

Ce modèle repose sur plusieurs concepts clés qui définissent les interactions entre les utilisateurs, les rôles, les objets, les opérations et les permissions. Comprendre ces termes est essentiel pour appréhender le fonctionnement et la mise en œuvre efficace du RBAC dans divers environnements informatiques.

Utilisateur: Dans le contexte du système, un utilisateur est défini comme une personne utilisant les fonctionnalités offertes.

Rôle: Le rôle se réfère à la position ou à la responsabilité qu'un individu peut occuper ou assumer au sein de l'organisation.

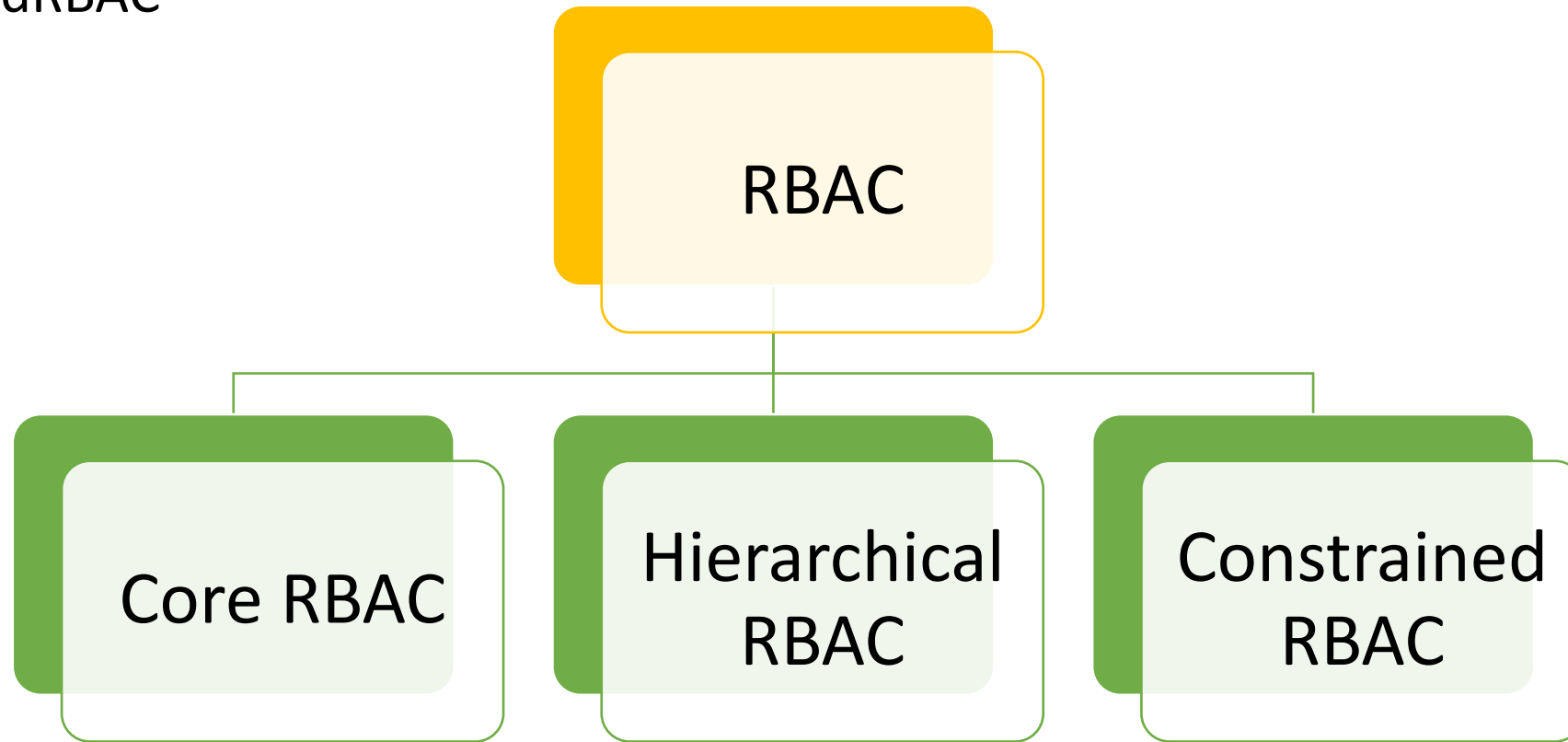
Objet: Tout élément représentant une ressource du système, tel qu'un fichier, une imprimante ou une base de données, est désigné comme un objet.

Opération: Une opération consiste en une action exécutable par le système en réponse à une demande de l'utilisateur.

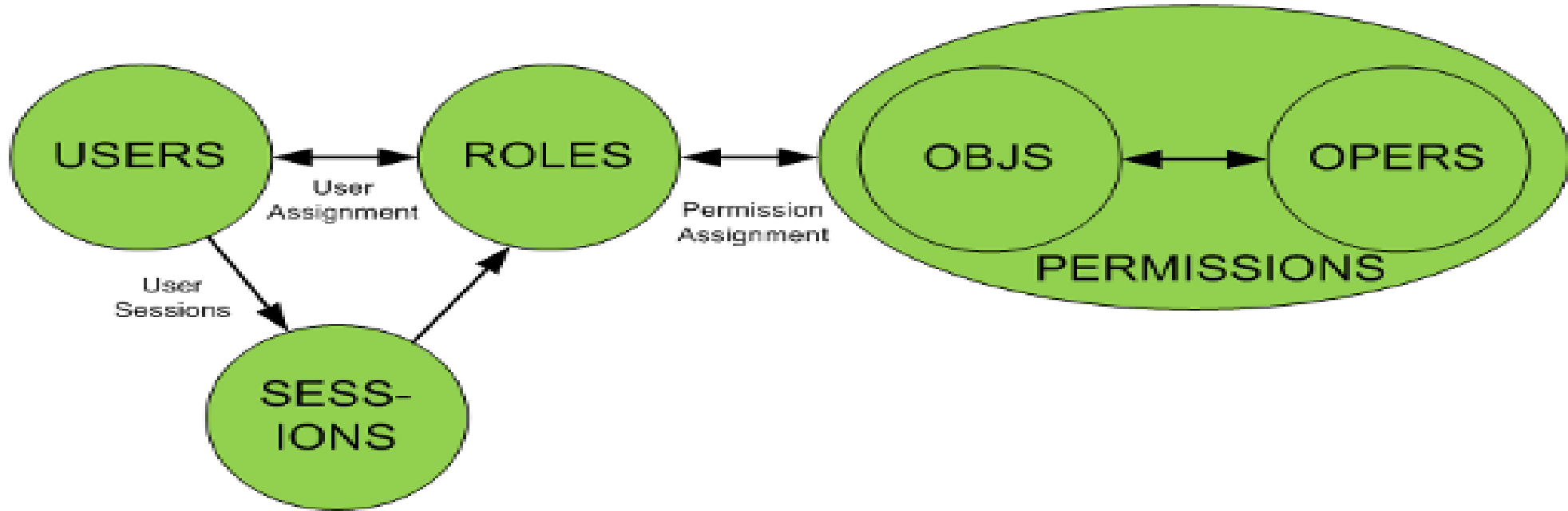
Permission: Une permission correspond à l'autorisation accordée pour effectuer une ou plusieurs opérations sur un objet donné.

Le modèle de contrôle d'accès basé sur le rôle (RBAC) (6)

Trois modèles de référence ont été définis dans le standard : CoreRBAC, HierarchicalRBAC et le ConstrainedRBAC



Core RBAC



Dans ce modèle de référence, on attribue des rôles aux utilisateurs, et ces rôles déterminent les autorisations qu'ils ont concernant divers objets et opérations au sein d'un système. Les sessions sont associées aux utilisateurs, indiquant des instances individuelles d'interaction de l'utilisateur avec le système. Le modèle RBAC est une approche courante pour gérer les autorisations des utilisateurs au sein des systèmes logiciels, offrant un moyen de restreindre l'accès au système aux utilisateurs autorisés.

HierarchicalRBAC

Le HierarchicalRBAC introduit des hiérarchies de rôles permettant de structurer les rôles afin de refléter les niveaux d'hiérarchie dans une organisation.

- Une hiérarchie de rôles définit des relations d'héritage entre les rôles.
- Un rôle $r1$ hérite d'un rôle $r2$ si tous les privilèges de $r2$ sont aussi des privilèges de $r1$.
- Le standard définit deux types de hiérarchies de rôles : des hiérarchies de rôles générales et des hiérarchies de rôles limitées.
- Une hiérarchie de rôles générale est un ordre partiel arbitraire représentant l'ensemble des rôles R . Cette hiérarchie supporte le concept d'héritage multiple qui offre la possibilité d'hériter des permissions de deux ou plusieurs rôles dans le système. De manière formelle, nous définissons un ordre partiel noté $\geq$ sur l'ensemble des rôles tel que :
 - étant donné deux rôles $r1$ et $r2$, $r1 \geq r2$ implique **$\text{authorized-permissions}(r2) \subseteq \text{authorized-permissions}(r1)$ et $\text{authorized-users}(r1) \subseteq \text{authorized-users}(r2)$** .
- La hiérarchie des rôles limitée impose la définition des rôles sous forme d'un arbre plus simple où un rôle ne peut avoir qu'un seul descendant.

HierarchicalRBAC (2)

Prenons un exemple pour illustrer cela. Supposons que nous ayons deux rôles dans un système de gestion de contenu : "Éditeur" et "Administrateur".

Le rôle "Éditeur" a des permissions pour créer, modifier et supprimer des articles sur le site web.

Le rôle **Administrateur** a des permissions pour effectuer toutes les actions des éditeurs ainsi que des permissions supplémentaires pour gérer les utilisateurs et les paramètres du site.

Dans cet exemple, nous avons un ordre partiel où le rôle "Administrateur" est supérieur au rôle **Éditeur**. Cela signifie que toutes les permissions autorisées pour un éditeur sont également autorisées pour un administrateur. De plus, tous les utilisateurs autorisés en tant qu'éditeurs seront également autorisés en tant qu'administrateurs.

En d'autres termes, si $r1$ représente le rôle "Administrateur" et $r2$ représente le rôle "Éditeur", alors $r1 \geq r2$ implique que les permissions autorisées pour "Éditeur" ($\text{authorized-permissions}(r2)$) doivent être un sous-ensemble des permissions autorisées pour "Administrateur" ($\text{authorized-permissions}(r1)$). De même, les utilisateurs autorisés en tant qu'éditeurs ($\text{authorized-users}(r2)$) doivent être un sous-ensemble des utilisateurs autorisés en tant qu'administrateurs ($\text{authorized-users}(r1)$).

ConstrainedRBAC

Le ConstrainedRBAC (Contrôle d'Accès Basé sur les Rôles avec Contraintes) introduit des règles spécifiques lors de l'attribution des rôles aux utilisateurs et lors de l'activation de ces rôles. Les contraintes de séparation de droit, également connues sous le nom de Separation of Duty Constraints (SoD), sont l'un des aspects clés de ce modèle.

Ces contraintes sont mises en place dans le but d'empêcher les utilisateurs d'acquérir un niveau excessif d'autorité ou de privilèges, et également de limiter les conflits potentiels qui pourraient survenir entre les politiques de sécurité du système.

ConstrainedRBAC (2)

Par exemple, considérons un système dans lequel un utilisateur est autorisé à approuver les transactions financières et également à valider les demandes d'accès aux données sensibles. Ces deux responsabilités, bien que distinctes, peuvent entraîner des conflits d'intérêts ou des risques de fraude si elles sont détenues par la même personne.

Les contraintes de séparation de droit dans le cadre du ConstrainedRBAC pourraient alors être utilisées pour spécifier qu'un utilisateur ne peut pas avoir à la fois le pouvoir d'approuver les transactions financières et celui de valider les demandes d'accès aux données sensibles. En imposant cette contrainte, le système garantit que les responsabilités sont réparties de manière appropriée et que les risques de conflits d'intérêts ou de fraudes sont réduits.

ConstrainedRBAC (3)

Pour formaliser les contraintes de sécurité, deux types de relations ont été établis : la séparation de droits statique (Static Separation of Duty - SSD) et la séparation de droits dynamique (Dynamic Separation of Duty - DSD).

Les contraintes statiques (SSD) servent à limiter l'attribution des rôles aux utilisateurs. Une fois qu'un utilisateur se voit attribuer un rôle R1, il lui est interdit de recevoir d'autres rôles susceptibles de créer des conflits avec le rôle R1. Ces contraintes sont largement employées dans divers contextes pour garantir une distribution équilibrée des responsabilités et prévenir les conflits d'intérêts ou de privilèges.

Les contraintes dynamiques (DSD) visent à éviter les conflits lors de l'activation des rôles. Elles autorisent un utilisateur à activer simultanément un ou plusieurs rôles, à condition que cette activation ne génère pas de conflit. Par exemple, un utilisateur pourrait se voir accorder les rôles d'éditeur et d'administrateur en même temps, pour autant que ces deux rôles ne s'opposent pas dans le système. Les contraintes DSD sont particulièrement utiles pour permettre une gestion flexible des autorisations tout en préservant la sécurité du système.

Exemples SSD et DSD

Supposons que nous ayons un système de gestion de contenu en ligne avec deux types d'utilisateurs : les éditeurs et les administrateurs.

Contraintes de séparation des droits statique (SSD) :

Un éditeur peut modifier et publier du contenu sur le site.

Un administrateur peut non seulement modifier et publier du contenu, mais aussi gérer les utilisateurs et les autorisations.

Selon la contrainte SSD, un utilisateur qui est désigné comme éditeur ne peut pas se voir attribuer le rôle d'administrateur simultanément. Cela garantit une séparation claire des responsabilités et évite les conflits potentiels entre les tâches éditoriales et administratives.

Contraintes de séparation des droits dynamique (DSD) :

Un utilisateur peut être à la fois modérateur et administrateur.

Un modérateur a le pouvoir de gérer les commentaires et le contenu utilisateur, tandis qu'un administrateur a des privilèges plus étendus, y compris la gestion des utilisateurs et des paramètres du site.

La contrainte DSD permet à un utilisateur d'activer simultanément les rôles de modérateur et d'administrateur, tant que ces rôles ne présentent pas de conflits fonctionnels. Par exemple, un utilisateur pourrait être autorisé à modérer les commentaires tout en ayant des droits d'administration limités, mais il ne pourrait pas être à la fois modérateur et éditeur, car ces deux rôles pourraient entrer en conflit dans la gestion du contenu.